电子电路设计实验 II 选题 A

信息与电子工程实验教学中心

二〇二五年二月

目录

一、基于 Arduino 的函数信号发生器 3
1、实验项目背景与目标 3
2、实验项目具体目标 3
3、项目步骤 3
4、所需材料与工具 3
5、硬件电路设计 3
6、软件编程 4
7、系统测试与调试 5
8、评估标准 5
9、注意事项 5
10、教学计划 5
二、基于 Arduino 的简易万用表制作 6
1、实验项目背景与目标 6
2、实验项目具体目标 6
3、项目步骤 6
4、所需材料与工具 6
5、硬件电路设计 6
6、软件编程 7
7、系统测试与调试 8
8、评估标准 8
9、注意事项 8
10、教学计划 8
三、 Arduino 温度监测与控制实验 9
1、实验项目背景与目标 9
2、实验项目具体目标 9
3、项目步骤 9
4、所需材料与工具 9
5、硬件电路设计 9
6、软件编程 10
7、系统测试与调试 10
8、评估标准 11
9、注意事项 11
10、教学计划 11
四、 Arduino 倒计时定时器的设计、制作与调试 12
1、实验项目背景与目标 12
2、实验项目具体目标 12
3、项目步骤 12
4、所需材料与工具 12
5、硬件电路设计 13
6、软件编程 13
7、系统测试与调试 14
8、评估标准 14

9、注意事项 14
10、教学计划 14
五、 Arduino 实时音频处理实验 15
1、实验项目背景与目标 15
2、实验项目具体目标 15
3、项目步骤 15
4、所需材料与工具 15
5、硬件电路设计 16
6、软件编程 16
7、系统测试与调试 17
8、评估标准 17
9、注意事项 17
10、教学计划 17
六、 Arduino 多功能数字时钟的设计与制作 18
1、实验项目背景与目标 18
2、实验项目具体目标 18
3、项目步骤 18
4、所需材料与工具 18

一、基于 Arduino 的函数信号发生器

1、实验项目背景与目标

背景：

在电子工程领域，函数信号发生器是一种重要的测试设备，它能够产生多种波形（如方波、三角波、正弦波等），广泛应用于电子电路设计、通信系统测试、音频信号处理等多个方面。随着微控制器技术的快速发展，利用 Arduino 等开源硬件平台设计并实现函数信号发生器成为可能，这不仅降低了成本，还增加了设计的灵活性和可扩展性。

目标：

- 掌握 Arduino 平台的基本使用方法，包括硬件连接、编程环境配置及代码上传。
- 理解并实现 DAC（数字模拟转换器）的基本原理，用于生成模拟信号。
- 学习如何通过编程控制产生方波、三角波和正弦波三种基本波形。
- 培养学生的实践动手能力和问题解决能力，通过实际项目加深对电子电路设计与信号处理的理解。

2、实验项目具体目标

1. 成功搭建基于 Arduino 的函数信号发生器硬件电路。
2. 编写并调试出能够生成方波、三角波、正弦波的 Arduino 程序。
3. 通过实验验证，确保生成的波形符合预期，频率和幅度可调。
4. 分析不同波形产生的原理，理解其在信号处理中的应用。

3、项目步骤

1. **理论学习**：复习 Arduino 基础知识，了解 DAC 工作原理，学习波形生成算法。
2. **硬件准备**：根据设计方案准备所需元器件，搭建电路。
3. **软件编程**：编写 Arduino 代码，实现波形生成功能。
4. **系统集成**：将硬件与软件结合，进行初步测试。
5. **调试与优化**：根据测试结果调整电路参数和程序代码，优化性能。
6. **评估与总结**：完成实验报告，分析实验数据，总结经验教训。

4、所需材料与工具

- Arduino Uno 板
- DAC 模块
- 面包板及连接线
- 电位器（用于调节幅度）
- 示波器（用于波形观察）
- 电阻、电容等辅助元件
- Arduino IDE 软件

5、硬件电路设计

- **Arduino 与 DAC 连接**：将 DAC 模块的输入端连接到 Arduino 的数字 I/O 口，用于

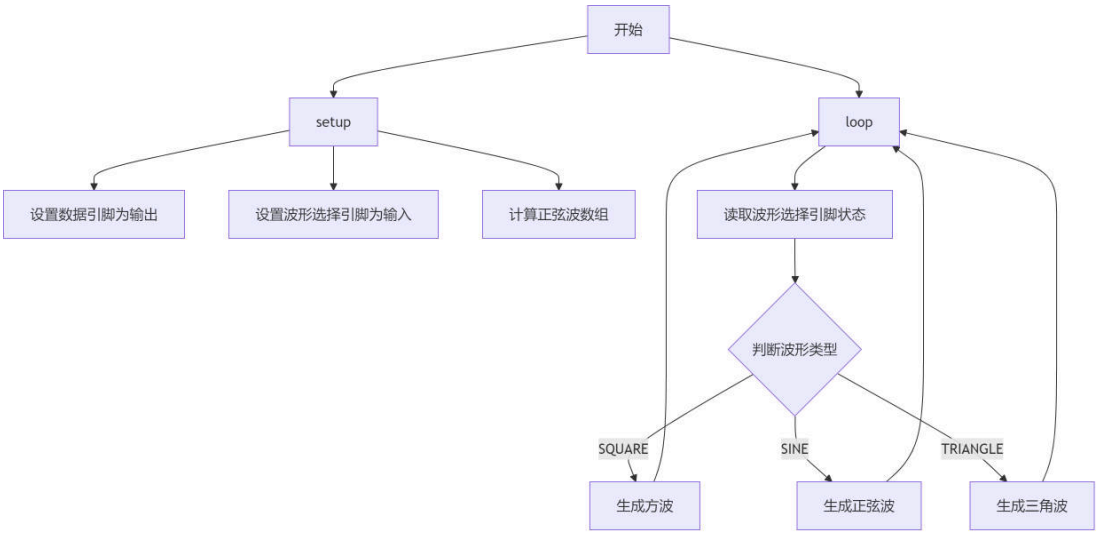
发送控制信号；输出端连接至示波器，作为信号输出。

- **幅度调节**：使用电位器连接至 Arduino 的模拟输入口，通过读取电位器的值来动态调整波形幅度。
- **电源管理**：确保所有元件正确供电，Arduino 通过 USB 或外部电源供电。
- **电路设计图**：详细的电路设计图，包括元件连接、引脚分配等

6、软件编程

- **初始化**：设置 DAC 模块的相关参数，配置 Arduino 的 I/O 口。
- **波形生成算法**：
  - **方波**：利用 PWM（脉宽调制）或直接输出高低电平实现。
  - **三角波**：通过逐步增加（或减少）DAC 的输出值模拟。
  - **正弦波**：使用查表法或实时计算（如利用正弦函数）生成。
- **幅度控制**：读取电位器的模拟值，转换为相应的幅度参数。
- **波形切换**：监听按钮开关的状态变化，根据用户操作切换波形类型。
- **循环输出**：在主循环中持续输出所选波形，根据用户输入切换波形类型。

流程图提供了代码执行的高层次视图，有助于理解代码的结构和逻辑。



流程图说明:

- **开始**：程序的入口点。
- **setup()**：初始化程序运行环境。
  - **设置数据引脚为输出**：配置用于 DAC 输出的数据引脚。
  - **设置波形选择引脚为输入**：配置用于选择波形类型的引脚。
  - **计算正弦波数组**：使用正弦函数计算并填充正弦波值的数组。
- **loop()**：主循环，程序持续运行。
  - **读取波形选择引脚状态**：读取波形选择引脚的状态以确定当前的波形类型。
  - **判断波形类型**：根据读取的状态，判断当前选择的波形类型。
    - **生成方波**：如果选择的是方波，执行方波生成函数。
    - **生成正弦波**：如果选择的是正弦波，执行正弦波生成函数。
    - **生成三角波**：如果选择的是三角波，执行三角波生成函数。
- 每个波形生成函数完成后，返回主循环继续读取波形选择引脚状态。

7、系统测试与调试

- **功能验证**：分别测试方波、三角波、正弦波的输出，确认波形正确无误。
- **幅度调节测试**：调整电位器，观察波形幅度的变化是否符合预期。
- **稳定性测试**：长时间运行系统，检查波形输出是否稳定，有无明显漂移或噪声。
- **故障排除**：针对测试中发现的问题，分析原因并采取相应措施解决。

8、评估标准

- 电路设计：原理图、PCB 版图设计合理性
- 波形生成的准确性（与理论波形对比）。
- 代码结构清晰，注释完整，易于理解和维护。
- 波形准确性：生成的波形需与理论波形高度一致。
- 幅度调节：幅度调节的灵活性和稳定性。
- 系统稳定性：长时间运行无故障，波形输出稳定。
- 代码质量：程序结构清晰，注释充分，易于理解和维护。
- 创新能力：是否进行功能扩展或性能优化。
- 实验报告内容完整，包括设计思路、实现过程、测试结果及分析。

9、注意事项

- 在连接电路前，确保所有元件电压兼容，避免损坏硬件。
- 使用示波器时，注意选择合适的探头衰减比和时基设置，以保护仪器和确保测量准确。
- 编程时注意内存管理和性能优化，避免程序运行缓慢或崩溃。
- 实验过程中注意安全，遵守实验室规章制度。

10、教学计划

- **理论讲解**（2 课时）：介绍基本原理及所需元器件。
- **电路设计与搭建**（10 课时）：指导学生设计电路图，分组进行硬件搭建，包括 AD 软件学习。
- **软件编程**（4 课时）：讲解编程思路，学生编写并调试代码。
- **系统测试与调试**（4 课时）：学生自行测试系统，记录数据，分析问题。
- **总结与展示**（2 课时）：学生分享项目成果，讨论遇到的问题及解决方案，教师点评。

二、基于 Arduino 的简易万用表制作

1、实验项目背景与目标

背景：

万用表是电子工程和实验室中常用的测量工具，能够测量电压、电流、电阻等多种电学参数。随着微控制器技术的发展，利用 Arduino 等开源硬件平台制作简易万用表成为可能。这种自制的万用表不仅具有教育意义，还能让学生深入理解电子测量的原理。

目标：

- 让学生掌握 Arduino 平台的基本使用方法，包括硬件连接、编程及数据采集。
- 让学生了解电压、电流、电阻等基本电学参数的测量方法。
- 通过实践，使学生能够制作出一个能够简单测量电压、电流和电阻的万用表。
- 培养学生的实践动手能力和问题解决能力。

2、实验项目具体目标

- 成功搭建基于 Arduino 的简易万用表硬件电路。
- 编写并调试出能够准确测量电压、电流和电阻的 Arduino 程序。
- 通过实验验证，确保万用表的测量结果与标准值相符，且具有一定的精度。
- 分析测量误差的来源，并提出改进措施。

3、项目步骤

1. **理论学习**：复习电子测量基础知识，了解 Arduino 平台及其编程环境。
2. **电路设计**：根据测量需求，设计硬件电路图，包括电压测量、电流测量和电阻测量部分。
3. **材料准备**：根据电路设计，准备所需的电子元件和工具。
4. **硬件组装**：按照电路图，将电子元件组装到面包板或 PCB 上，并连接 Arduino。
5. **软件编程**：编写 Arduino 程序，实现测量功能，包括数据读取、处理和显示。
6. **系统测试**：对制作的万用表进行功能测试，确保其能够准确测量电压、电流和电阻。
7. **调试与优化**：根据测试结果，对硬件电路和软件程序进行调试和优化。
8. **总结与展示**：撰写实验报告，展示制作过程和成果，分享经验和心得。

4、所需材料与工具

- Arduino 开发板（如 Arduino Uno）
- 面包板或 PCB 板
- 电子元件：电阻、电容、二极管、电位器等
- 显示屏（如 LCD1602）
- 万用表（用于校准和测试）

5、硬件电路设计

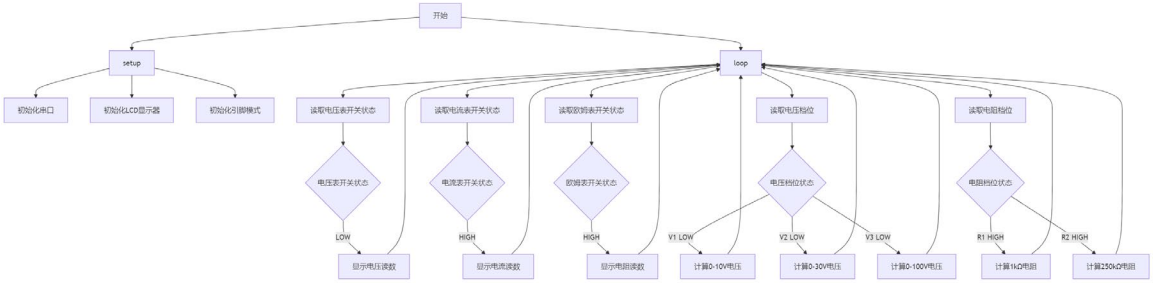
- **电压测量**：利用 Arduino 的模拟输入引脚，通过分压电路测量电压。
- **电流测量**：使用取样电阻（1 欧姆）和 ADC 测量电压降来计算电流。
- **电阻测量**：利用欧姆定律，通过测量电压和电流计算电阻值。

- **显示屏连接**：将显示屏连接到 Arduino 的数字引脚，用于显示测量结果。
- **按钮**：选择测量模式

6、软件编程

- **初始化**：配置 Arduino 引脚，初始化显示屏。
- **数据读取**：读取模拟输入引脚的电压值，或通过传感器获取电流值。
- **数据处理**：根据测量原理，计算电压、电流或电阻的实际值。
- **数据显示**：将计算结果发送到显示屏进行显示。
- **用户交互**：设计简单的用户交互界面，如按钮选择测量模式等。

流程图提供了代码执行的高层次视图，有助于理解代码的结构和逻辑。每个决策点（如开关状态和档位状态）都会导致不同的操作，如读取和显示电压、电流或电阻的值。



流程图说明:

- **开始**：程序的入口点。
- **setup()**：初始化程序运行环境。
  - **初始化串口**：设置串口波特率。
  - **初始化 LCD 显示器**：初始化 LCD 显示器。
  - **初始化引脚模式**：设置电压表开关、电流表开关、欧姆表开关、电压档位和电阻档位的引脚模式。
- **loop()**：主循环，程序持续运行。
  - **读取电压表开关状态**：读取电压表开关的状态。
  - **读取电流表开关状态**：读取电流表开关的状态。
  - **读取欧姆表开关状态**：读取欧姆表开关的状态。
  - **读取电压档位**：读取电压档位的状态。
  - **读取电阻档位**：读取电阻档位的状态。
  - **判断电压表开关状态**：
    - **LOW**：如果是 LOW，显示电压读数。
  - **判断电流表开关状态**：
    - **HIGH**：如果是 HIGH，显示电流读数。
  - **判断欧姆表开关状态**：
    - **HIGH**：如果是 HIGH，显示电阻读数。
  - **判断电压档位状态**：
    - **V1 LOW**：如果是 V1 LOW，计算 0-10V 电压。
    - **V2 LOW**：如果是 V2 LOW，计算 0-30V 电压。
    - **V3 LOW**：如果是 V3 LOW，计算 0-100V 电压。
  - **判断电阻档位状态**：
    - **R1 HIGH**：如果是 R1 HIGH，计算 1kΩ电阻。

- **R2 HIGH**：如果是 R2 HIGH，计算 250kΩ电阻。

7、系统测试与调试

- **校准**：使用已知电压、电流和电阻值对万用表进行校准，确保测量准确。
- **功能测试**：分别测试电压、电流和电阻测量功能，记录测试结果。
- **稳定性测试**：在长时间内对万用表进行连续测量，观察其稳定性和重复性。
- **故障排查**：针对测试中出现的问题，分析原因并采取相应措施进行解决。

8、评估标准

- **万用表功能**：能够准确测量电压、电流和电阻，且测量范围符合设计要求。
- **测量精度**：与标准值相比，测量误差在允许范围内。
- **稳定性与重复性**：在长时间内测量结果保持稳定，且重复测量时误差较小。
- **创新性与实用性**：在设计过程中有创新思路，且制作的万用表具有实际应用价值。
- **报告质量**：实验报告内容完整、条理清晰，能够准确反映制作过程和成果。

9、注意事项

- 在使用烙铁等工具时，注意安全，避免烫伤或触电。
- 在连接电路时，确保电源极性正确，避免损坏元件。
- 在测量高电压或大电流时，需采取适当的保护措施，确保人身安全。
- 在编程时，注意代码的逻辑性和可读性，便于后续调试和修改。

10、教学计划

- **理论讲解**（2 课时）：介绍基本原理及所需元器件。
- **电路设计与搭建**（10 课时）：指导学生设计电路图，分组进行硬件搭建，包括 AD 软件学习。
- **软件编程**（4 课时）：讲解编程思路，学生编写并调试代码。
- **系统测试与调试**（4 课时）：学生自行测试系统，记录数据，分析问题。
- **总结与展示**（2 课时）：学生分享项目成果，讨论遇到的问题及解决方案，教师点评。

三、 Arduino 温度监测与控制实验

1、实验项目背景与目标

背景:

随着物联网技术的快速发展，温度监测与控制系统在日常生活和工业生产中扮演着越来越重要的角色。Arduino 作为一款开源的硬件平台，其简单易用、功能强大的特点使其成为初学者学习电子电路设计和编程的理想选择。

目标:

本实验旨在通过设计并实现一个基于 Arduino 的温度监测与控制系统，使学生掌握温度传感器的使用、信号调理、数据采集与处理以及控制算法的应用。同时，培养学生的实践动手能力、问题解决能力和创新思维。

2、实验项目具体目标

- 熟悉 Arduino 开发环境及基本编程方法。
- 掌握温度传感器的工作原理及接口电路设计。
- 实现温度数据的实时采集与显示。
- 根据设定温度阈值，实现温度控制逻辑。

3、项目步骤

1. **理论学习**：复习温度传感器的工作原理、Arduino 编程基础及控制算法。
2. **硬件准备**：根据设计需求，准备所需的电子元件及 Arduino 开发板。
3. **电路设计**：设计温度监测与控制电路的原理图，并搭建实物电路。
4. **软件编程**：编写 Arduino 程序，实现温度数据采集、处理及控制逻辑。
5. **系统集成**：将硬件电路与软件程序结合，进行初步测试。
6. **调试与优化**：根据测试结果，对系统进行调整和优化。
7. **成果展示**：撰写实验报告，展示项目成果及心得体会。

4、所需材料与工具

- Arduino 开发板（如 Arduino Uno），USB 数据线
- 温度传感器（如 LM35）、继电器、加热电阻、普通电阻、电容等辅助元件
- 自制 PCB 板
- 直流电源
- 万用表、示波器等测试工具
- 电脑及 Arduino IDE 软件

5、硬件电路设计

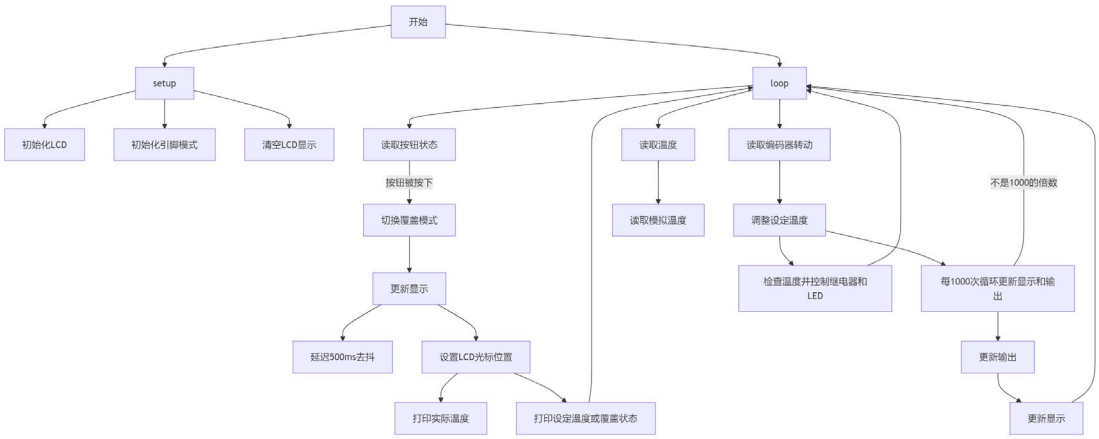
- **温度传感器接口电路**：根据传感器类型设计相应的接口电路，确保传感器能够正常工作并输出准确的温度信号。
- **信号处理电路**（如需要）：对传感器输出的信号进行放大、滤波等处理，以满足后续数据采集的需求。
- **控制输出电路**：根据控制需求设计相应的输出电路，如继电器控制、PWM 输出等。

- **电源电路**：为整个系统提供稳定的供电。

6、软件编程

- **初始化**：配置 Arduino 引脚、初始化传感器及通信接口。
- **数据采集**：编写程序读取温度传感器的数据，并进行必要的转换和处理。
- **控制逻辑**：根据设定的温度阈值，编写控制算法实现温度调节。
- **数据显示**：通过串口监视器或 LCD 显示屏实时显示当前温度及控制状态。

流程图提供了代码执行的高层次视图，有助于理解代码的结构和逻辑。每个步骤都对应代码中的一个操作或函数调用，显示了程序如何读取输入、处理数据并输出结果。



流程图说明:

- **开始**：程序的入口点。
- **setup()**：初始化程序运行环境。
  - **初始化 LCD**：设置 LCD 的尺寸和引脚。
  - **初始化引脚模式**：设置 LED、继电器、编码器和按钮的引脚模式。
  - **清空 LCD 显示**：清除 LCD 上的所有内容。
- **loop()**：主循环，程序持续运行。
  - **读取温度**：从模拟引脚读取温度。
  - **读取按钮状态**：检查按钮是否被按下。
    - **按钮被按下**：切换覆盖模式并更新显示。
  - **延迟 500ms 去抖**：为了避免误操作，延迟 500 毫秒。
  - **读取编码器转动**：读取编码器的转动状态。
  - **调整设定温度**：根据编码器的转动调整设定温度。
  - **每 1000 次循环更新显示和输出**：每 1000 次循环更新一次 LCD 显示和继电器、LED 的输出状态。
    - **更新输出**：根据温度控制继电器和 LED。
    - **更新显示**：在 LCD 上显示实际温度和设定温度或覆盖状态。
- **读取模拟温度**：读取模拟引脚的值并转换为温度值。
- **检查温度并控制继电器和 LED**：根据实际温度和设定温度控制继电器和 LED 的状态。

7、系统测试与调试

- **功能测试**：分别测试温度采集、数据显示及控制输出等功能是否正常。
- **性能测试**：在不同环境温度下测试系统的稳定性和准确性。

- **故障排查**：针对测试中出现的问题，分析原因并采取相应措施进行解决。
- **优化建议**：根据测试结果提出改进意见，如优化控制算法、提高系统响应速度等。

8、评估标准

- **功能完整性**：系统是否实现了所有设定的功能。
- **性能稳定性**：系统在不同环境下的稳定性和准确性如何。
- **代码质量**：程序结构是否清晰、逻辑是否合理、注释是否完善。
- **报告质量**：实验报告是否内容完整、条理清晰、分析深入。
- **创新点**：项目中有无独特的创新点或改进之处。

9、注意事项

- 在进行电路设计时，要确保元件的规格和参数符合设计要求，避免损坏元件或造成安全隐患。
- 在编程时，要注意代码的可读性和可维护性，便于后续调试和修改。
- 在测试过程中，要严格遵守安全操作规程，避免触电或烫伤等意外事故。

10、教学计划

- **理论讲解**（2 课时）：介绍基本原理及所需元器件。
- **电路设计与搭建**（10 课时）：指导学生设计电路图，分组进行硬件搭建，包括 AD 软件学习。
- **软件编程**（4 课时）：讲解编程思路，学生编写并调试代码。
- **系统测试与调试**（4 课时）：学生自行测试系统，记录数据，分析问题。
- **总结与展示**（2 课时）：学生分享项目成果，讨论遇到的问题及解决方案，教师点评。

四、 Arduino 倒计时定时器的设计、制作与调试

1、实验项目背景与目标

背景：

倒计时定时器在日常生活和工业生产中有着广泛的应用，如厨房烹饪、体育比赛、实验室实验等。通过本项目，学生将学习如何利用 Arduino 开源硬件平台，结合电子电路设计和编程技能，设计并制作一个实用的倒计时定时器。

目标：

- 让学生掌握 Arduino 平台的基本使用方法，包括硬件连接、编程环境配置及代码编写。
- 让学生了解倒计时定时器的工作原理，学会如何设计其硬件电路和编写控制程序。
- 通过实践，使学生能够独立完成一个基于 Arduino 的倒计时定时器的设计、制作与调试过程。
- 培养学生的实践动手能力、问题解决能力和团队协作能力。

2、实验项目具体目标

- 设计并制作出一个能够准确倒计时的 Arduino 定时器。
- 定时器能够设置任意时间长度，并在倒计时结束时发出提示信号。
- 通过编程实现定时器的各种功能，如开始、暂停、复位等。
- 对制作的定时器进行系统测试，确保其准确性和稳定性。

3、项目步骤

1. **理论学习**：复习 Arduino 基础知识，了解倒计时定时器的工作原理及所需元器件。
2. **硬件准备**：根据设计方案准备所需元器件，包括 Arduino 板、显示器、按键、蜂鸣器等。
3. **电路搭建**：按照电路设计图连接元器件，构建倒计时定时器硬件电路。
4. **软件编程**：在 Arduino IDE 中编写程序，实现倒计时的设置、显示、控制及提示功能。
5. **系统集成**：将硬件电路与软件程序结合，进行初步测试与调试。
6. **系统优化**：根据测试结果调整电路参数和程序代码，优化定时器性能。
7. **评估与总结**：完成实验报告，分析实验数据，总结项目经验。

4、所需材料与工具

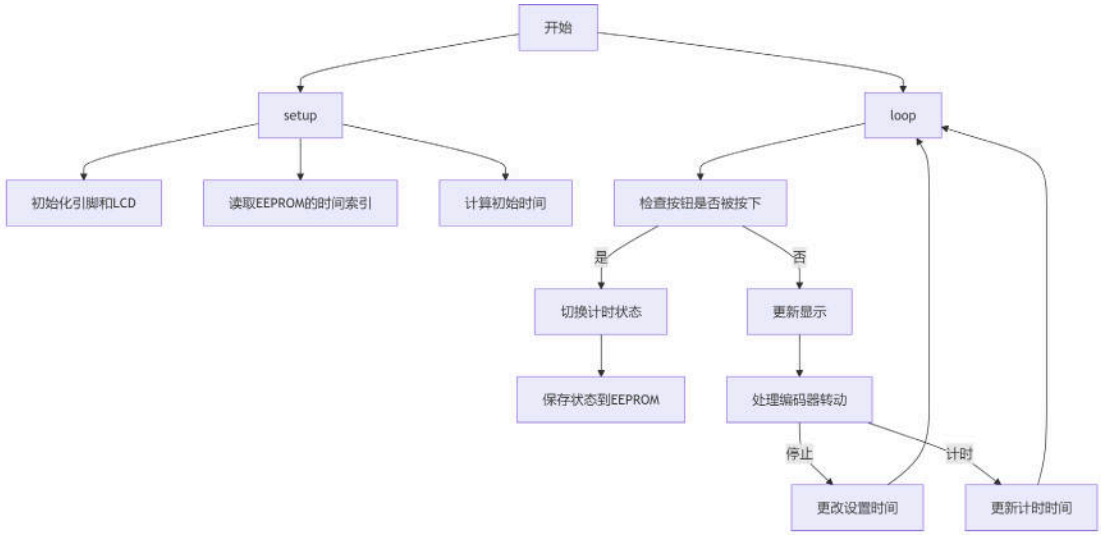
- Arduino 板（如 Arduino Uno）
- 显示器（如 7 段数码管或 LCD 显示屏）
- 按键（用于设置时间、开始/暂停、复位等操作）
- 蜂鸣器（用于倒计时结束时的提示）
- 电阻、电容等辅助元件
- 面包板及连接线
- 电脑及 Arduino IDE 软件

5、硬件电路设计

- **显示器连接**：将显示器的数据线和控制线连接到 Arduino 的相应引脚，用于显示倒计时时间。
- **按键连接**：将按键的一端连接到 Arduino 的数字输入口，另一端接地，通过编程实现按键功能。
- **蜂鸣器连接**：将蜂鸣器的一端连接到 Arduino 的数字输出口，另一端接地，当倒计时结束时控制蜂鸣器发声。
- **电源管理**：确保所有元件正确供电，Arduino 通过 USB 或外部电源供电，其他元件根据需要连接至适当的电源。
- **电路设计图**：详细的电路设计图，包括元件连接、引脚分配等

6、软件编程

- **初始化**：设置 Arduino 的 I/O 口，配置显示器和按键的工作参数。
- **时间设置**：通过按键输入设置倒计时时间，并在显示器上显示。
- **倒计时逻辑**：实现倒计时的计时逻辑，每秒更新一次显示时间。
- **控制功能**：实现开始、暂停、复位等控制功能，通过按键操作控制倒计时的进行。
- **提示功能**：当倒计时结束时，控制蜂鸣器发声提示用户。
  流程图提供了代码执行的主要步骤，有助于快速理解代码的结构和逻辑。



流程图说明:

- **开始**：程序的入口点。
- **setup()**：初始化程序运行环境，包括引脚模式和 LCD 显示器。
  - **初始化引脚和 LCD**: 设置所有需要的引脚为输入或输出模式，并初始化 LCD 显示器。
  - **读取 EEPROM 的时间索引**：从 EEPROM 中读取保存的时间索引。
  - **计算初始时间**：根据时间索引计算分钟和秒。
- **loop()**：主循环，程序持续运行。
  - **检查按钮是否被按下**：检测按钮是否被按下以切换计时状态。
    - **切换计时状态**：如果按钮被按下，切换计时状态。
    - **保存状态到 EEPROM**：将当前的计时状态保存到 EEPROM。
  - **更新显示**：根据当前的计时状态更新 LCD 显示。
  - **处理编码器转动**：处理编码器的转动来调整时间。

- **更改设置时间**：如果编码器转动，更改时间设置。
- **更新计时时间**：如果计时器正在运行，更新计时时间。

7、系统测试与调试

- **功能测试**：分别测试时间设置、显示、控制及提示功能，确保各项功能正常工作。
- **性能测试**：设置不同的倒计时时间进行测试，评估定时器的准确性和稳定性。
- **故障排查**：针对测试中出现的问题，分析原因并采取相应措施进行解决。

8、评估标准

- 电路设计：原理图、PCB 版图设计合理性
- 定时器能够准确设置和显示倒计时时间。
- 控制功能完善，能够实现开始、暂停、复位等操作。
- 倒计时结束时能够准确触发提示信号。
- 系统稳定性好，能够在不同环境条件下正常工作。
- 实验报告内容完整，包括设计思路、实现过程、测试结果及分析。

9、注意事项

- 在连接电路前，确保所有元件的电压和电流规格符合 Arduino 的要求，避免损坏硬件。
- 在编程时，注意代码的可读性和可维护性，便于后续修改和扩展。
- 在测试过程中，注意观察定时器的运行状态和显示结果，及时发现并解决问题。
- 遵守实验室的规章制度，保持工作区域的整洁和安全。

10、教学计划

- **理论讲解**（2 课时）：介绍基本原理及所需元器件。
- **电路设计与搭建**（10 课时）：指导学生设计电路图，分组进行硬件搭建，包括 AD 软件学习。
- **软件编程**（4 课时）：讲解编程思路，学生编写并调试代码。
- **系统测试与调试**（4 课时）：学生自行测试系统，记录数据，分析问题。
- **总结与展示**（2 课时）：学生分享项目成果，讨论遇到的问题及解决方案，教师点评。

五、 Arduino 实时音频处理实验

1、实验项目背景与目标

背景：

随着音频处理技术的不断发展，实时音频处理在音频信号处理、语音识别、音频合成等领域得到了广泛应用。Arduino 作为一款开源的硬件平台，其灵活性和易用性使得它成为学习和实验实时音频处理的理想选择。本实验旨在通过 Arduino 平台，结合相关音频处理模块，让学生深入了解实时音频处理的基本原理和实现方法。

目标：

- 让学生掌握 Arduino 平台的基本使用方法，包括硬件连接、编程环境配置及代码编写。
- 让学生了解实时音频处理的基本原理，学会如何采集、处理和输出音频信号。
- 通过实践，使学生能够独立完成一个基于 Arduino 的实时音频处理项目的设计、制作与调试。
- 培养学生的实践动手能力、问题解决能力和创新能力。

2、实验项目具体目标

- 应用音频软件生成测试信号。
- 对采集到的音频信号进行实时处理，如滤波、增益调整等。
- 将处理后的音频信号实时输出，通过扬声器或耳机进行监听。
- 编写相应的 Arduino 程序，实现音频信号的采集、处理和输出功能。

3、项目步骤

1. **理论学习**：复习 Arduino 基础知识，了解音频处理的基本原理及所需元器件。
2. **硬件准备**：根据设计方案准备所需元器件，包括 Arduino 板、音频处理模块（如麦克风模块、音频放大器模块）、扬声器或耳机等。
3. **电路搭建**：按照电路设计图连接元器件，构建实时音频处理硬件电路。
4. **软件编程**：在 Arduino IDE 中编写程序，实现音频信号的采集、处理和输出功能。
5. **系统集成**：将硬件电路与软件程序结合，进行初步测试与调试。
6. **系统优化**：根据测试结果调整电路参数和程序代码，优化音频处理效果。
7. **评估与总结**：完成实验报告，分析实验数据，总结项目经验。

4、所需材料与工具

- Arduino 板（如 Arduino Uno）
- 扬声器或耳机
- 电阻、电容等辅助元件
- 面包板及连接线
- 自制 PCB 板
- 电脑及 Arduino IDE 软件
- 音频测试工具（如音频信号发生器、示波器等）

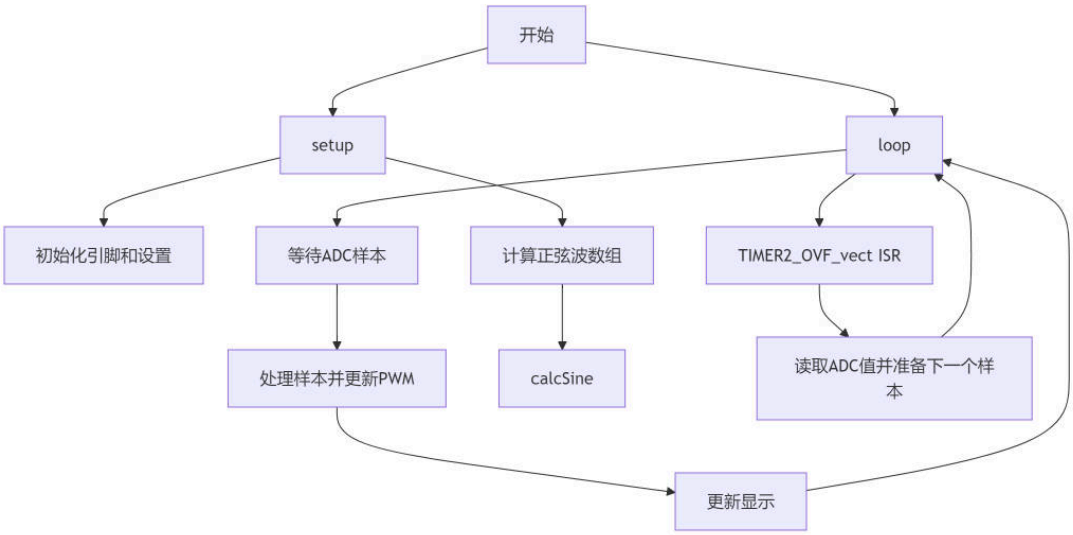
5、硬件电路设计

- **麦克风模块连接**：将麦克风模块的输出端连接到 Arduino 的模拟输入口，用于采集音频信号。
- **音频放大器模块连接**：将音频放大器模块的输入端连接到 Arduino 的数字输出口（或 PWM 口），用于输出处理后的音频信号。同时，将放大器模块的输出端连接到扬声器或耳机。
- **电源管理**：确保所有元件正确供电，Arduino 通过 USB 或外部电源供电，其他元件根据需要连接至适当的电源。
- **电路设计图**：详细的电路设计图，包括元件连接、引脚分配等

6、软件编程

- **初始化**：设置 Arduino 的 I/O 口，配置麦克风模块和音频放大器模块的工作参数。
- **音频处理**：对采集到的音频信号进行实时处理，如增益调整、滤波等。
- **音频输出**：将处理后的音频信号通过数字输出口（或 PWM 口）输出到音频放大器模块，驱动扬声器或耳机发声。
- **代码示例：**提供部分实时音频信号处理代码示例。

流程图，描述了 Arduino 代码的执行流程，该代码用于生成一个电压控制振荡器（VCO）的波形。流程图提供了代码执行的主要步骤，有助于快速理解代码的结构和逻辑。每个步骤都对应代码中的一个操作或函数调用，显示了程序如何读取输入、处理数据并输出结果。



流程图说明:

- **开始**：程序的入口点。
- **setup()**：初始化程序运行环境，包括引脚模式和 ADC 设置。
  - **计算正弦波数组**：计算正弦波值并存储在数组中。
  - **calcSine()**：计算正弦波数组。
- **loop()**：主循环，程序持续运行。
  - **等待 ADC 样本**：等待新的 ADC 样本准备就绪。
  - **处理样本并更新 PWM**：处理 ADC 样本，更新 PWM 输出。
  - **更新显示**：在 LCD 上显示当前的波形信息。
- **TIMER2_OVF_vect ISR**：Timer2 溢出中断服务例程，用于读取 ADC 值并准备下一

个样本。

7、系统测试与调试

- **功能测试**：分别测试音频采集、处理和输出功能，确保各项功能正常工作。
- **性能测试**：使用音频测试工具生成不同频率和幅度的音频信号，测试系统的响应和失真情况。
- **实时性测试**：在实时环境下测试系统的处理速度和延迟情况，确保满足实时音频处理的要求。
- **故障排查**：针对测试中出现的问题，分析原因并采取相应措施进行解决。

8、评估标准

- 电路设计：原理图、PCB 版图设计合理性
- 系统能够实时采集音频信号，并准确输出处理后的音频。
- 音频处理效果良好，无明显失真或噪声。
- 系统稳定性好，能够在长时间运行下保持正常工作。
- 实验报告内容完整，包括设计思路、实现过程、测试结果及分析。

9、注意事项

- 在连接电路前，确保所有元件的电压和电流规格符合 Arduino 的要求，避免损坏硬件。
- 在编程时，注意代码的可读性和可维护性，便于后续修改和扩展。
- 在测试过程中，注意观察系统的运行状态和输出效果，及时发现并解决问题。
- 使用音频测试工具时，注意操作规范和安全事项，避免对测试工具或人员造成损坏或伤害。

10、教学计划

- **理论讲解**（2 课时）：介绍基本原理及所需元器件。
- **电路设计与搭建**（10 课时）：指导学生设计电路图，分组进行硬件搭建，包括 AD 软件学习。
- **软件编程**（4 课时）：讲解编程思路，学生编写并调试代码。
- **系统测试与调试**（4 课时）：学生自行测试系统，记录数据，分析问题。
- **总结与展示**（2 课时）：学生分享项目成果，讨论遇到的问题及解决方案，教师点评。

六、 Arduino 多功能数字时钟的设计与制作

1、实验项目背景与目标

背景:

随着科技的不断发展，数字时钟已经成为我们日常生活中不可或缺的一部分。Arduino 作为一款开源的硬件平台，其灵活性和易用性使得它成为学习和实验数字时钟设计的理想选择。通过本实验，学生将能够深入了解数字时钟的工作原理，并掌握 Arduino 平台的基本使用方法。

目标:

- 让学生掌握 Arduino 平台的基本使用方法，包括硬件连接、编程环境配置及代码编写。
- 让学生了解数字时钟的基本原理，学会如何设计并制作一个多功能数字时钟。
- 通过实践，使学生能够独立完成一个基于 Arduino 的多功能数字时钟项目的设计、制作与调试。
- 培养学生的实践动手能力、问题解决能力和创新能力。

2、实验项目具体目标

- 设计并制作一个能够显示时间、日期和星期的多功能数字时钟。
- 实现闹钟功能，当达到设定时间时，能够通过蜂鸣器或 LED 进行提示。
- （可选）实现温度显示功能，通过温度传感器实时显示当前环境温度。
- 编写相应的 Arduino 程序，实现数字时钟的各项功能。

3、项目步骤

1. **理论学习**：复习 Arduino 基础知识，了解数字时钟的基本原理及所需元器件。
2. **硬件准备**：根据设计方案准备所需元器件，包括 Arduino 板、显示屏（如 OLED 或 LCD）、时钟模块（如 DS1302）、蜂鸣器、LED（可选）、温度传感器（可选）等。
3. **电路搭建**：按照电路设计图连接元器件，构建多功能数字时钟硬件电路。
4. **软件编程**：在 Arduino IDE 中编写程序，实现时间显示、闹钟设置与提示、温度显示（如选用）等功能。
5. **系统集成**：将硬件电路与软件程序结合，进行初步测试与调试。
6. **系统优化**：根据测试结果调整电路参数和程序代码，优化数字时钟的性能和用户体验。
7. **评估与总结**：完成实验报告，分析实验数据，总结项目经验。

4、所需材料与工具

- Arduino 板（如 Arduino Uno）
- 显示屏（OLED 或 LCD）
- 时钟模块
- 蜂鸣器
- LED
- 温度传感器（如 LM35）
- 电阻、电容等辅助元件
- 面包板及连接线

- 电脑及 Arduino IDE 软件

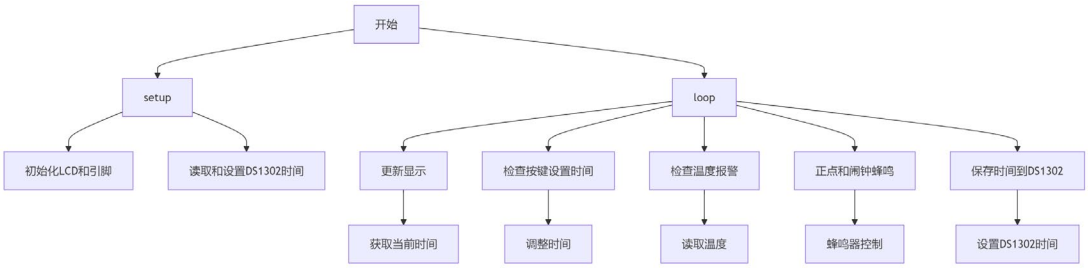
5、硬件电路设计

- **显示屏连接**：将显示屏的接口连接到 Arduino 的相应数字口，用于显示时间、日期和星期等信息。
- **时钟模块连接**：将时钟模块的接口连接到 Arduino 的 I2C 或 SPI 接口，用于获取当前时间信息。
- **蜂鸣器与 LED 连接**：将蜂鸣器和 LED 分别连接到 Arduino 的数字输出口，用于闹钟提示。
- **温度传感器连接**：将温度传感器的接口连接到 Arduino 的模拟输入口，用于获取当前环境温度信息。
- **电源管理**：确保所有元件正确供电，Arduino 通过 USB 或外部电源供电，其他元件根据需要连接至适当的电源。
- **电路设计图**：详细的电路设计图，包括元件连接、引脚分配等

6、软件编程

- **初始化**：配置 Arduino 引脚，初始化时钟模块、显示屏、温度传感器等。
- **时间显示**：从时钟模块读取当前时间，并在显示屏上显示。
- **闹钟设置**：通过按键设置闹钟时间，并存储在 Arduino 的内存中。
- **温度显示**：从温度传感器读取当前温度，并在显示屏上显示。
- **闹钟提醒**：当当前时间与闹钟时间相匹配时，启动蜂鸣器进行提醒。
- **用户交互**：设计简洁的用户界面，方便用户设置时间与闹钟。

流程图提供了代码执行的主要步骤，有助于快速理解代码的结构和逻辑。每个步骤都对应代码中的一个操作或函数调用，显示了程序如何读取输入、处理数据并输出结果。



流程图说明：

- 开始：程序的入口点。
- setup()：初始化程序运行环境，包括 LCD 显示器和引脚模式。
    - 初始化 LCD 和引脚：设置 LCD 显示和所有相关的输入输出引脚。
    - 读取和设置 DS1302 时间：从 DS1302 实时时钟芯片读取时间，并在需要时设置初始时间。
- loop()：主循环，程序持续运行。
    - 更新显示：在 LCD 上显示当前时间和其他信息。
    - 检查按键设置时间：检查是否有按键操作来调整时间。
        - 调整时间：根据按键输入调整时间。
    - 检查温度报警：检查温度是否超过设定的报警温度。
        - 读取温度：从温度传感器读取当前温度。
    - 正点和闹钟蜂鸣：控制蜂鸣器在正点和设定的闹钟时间发出声音。
        - 蜂鸣器控制：根据时间条件控制蜂鸣器。

    - 保存时间到 DS1302：在程序运行过程中定期将当前时间保存到 DS1302 芯片中。
        - 设置 DS1302 时间：更新 DS1302 芯片上的时间。

7、系统测试与调试

- **功能测试**：分别测试时间显示、闹钟提示和温度显示（如选用）功能，确保各项功能正常工作。
- **性能测试**：测试数字时钟的准确性和稳定性，确保时间显示无误且系统能够长时间稳定运行。
- **故障排查**：针对测试中出现的问题，分析原因并采取相应措施进行解决。

8、评估标准

- 电路设计：原理图、PCB 版图设计合理性
- 系统能够准确显示时间、日期和星期，且闹钟功能正常。
- 温度显示准确，无明显误差。
- 系统稳定性好，能够在长时间运行下保持正常工作。
- 实验报告内容完整，包括设计思路、实现过程、测试结果及分析。

9、注意事项

- 在连接电路前，确保所有元件的电压和电流规格符合 Arduino 的要求，避免损坏硬件。同时，检查所有连接线是否接触良好，避免出现短路或断路现象。
- 在编程时，注意代码的逻辑性和可读性，遵循良好的编程规范。在修改代码时，要小心谨慎，避免引入新的错误。
- 在测试过程中，注意观察系统的运行状态，及时发现并解决问题。若遇到复杂问题，可以寻求教师或同学的帮助，共同解决。

10、教学计划

- **理论讲解**（2 课时）：介绍基本原理及所需元器件。
- **电路设计与搭建**（10 课时）：指导学生设计电路图，分组进行硬件搭建，包括 AD 软件学习。
- **软件编程**（4 课时）：讲解编程思路，学生编写并调试代码。
- **系统测试与调试**（4 课时）：学生自行测试系统，记录数据，分析问题。
- **总结与展示**（2 课时）：学生分享项目成果，讨论遇到的问题及解决方案，教师点评。